Lezione — CALC 03: Linguaggio macchina

Corso: Calcolatori Elettronici T
Materiale: `03_Linguaggio_macchina.pdf` (51 sl., ed. 2024/25) · prova 2023-01-17 (testo + soluzione ufficiale)
Prerequisiti: [[lezione_02_mapping_decodifica]] (mappa, CS, banchi): il codice scrive e legge proprio agli indirizzi che lì hai deciso
Consegna servita: d) «Scrivere il codice dell'interrupt handler» — la parte che legge e scrive i dispositivi

---

Obiettivo

Saper scrivere le poche righe di assembler DLX che, all'esame, **leggono da un dispositivo e scrivono in memoria** agli indirizzi fissati nella consegna b), scegliendo l'istruzione giusta per la dimensione e il segno del dato.

1. Il DLX visto dal programmatore: 32 registri e niente altro

La decisione. Ogni dato passa da un registro: il DLX ha **32 registri GP da 32 bit**, `R0…R31`, con **`R0 = 0`** sempre [fonte: 03 sl. 14–15]. Le operazioni aritmetico-logiche si fanno **solo tra registri** («non tra registri e memoria») [sl. 14]: per sommare due word in memoria si caricano in due registri, si somma, si riscrive.

Il perché. È la scelta RISC: istruzioni semplici e molti registri, perché la memoria è lenta [sl. 2, 5]. Ogni istruzione è lunga **32 bit**, con campi in posizioni fisse [sl. 14], e la rete di controllo (moduli 07–08) resta semplice.

La notazione. Tre formati [sl. 20]:

Formato	Campi	Chi lo usa
I	OpCode 6 · RS1 5 · RS2/Rd 5 · **Immediato 16**	load, store, branch, ALU con immediato, `LHI`
R	OpCode 6 · RS1 · RS2 · Rd · OpCode ext. 11	ALU tra registri, Sx tra registri
J	OpCode 6 · **Immediato/offset 26**	`J`, `JAL`

Da qui viene tutto il resto: **nell'istruzione c'è spazio per un numero di 16 bit**, non di 32.

Altri fatti fissi [sl. 14]: **non c'è lo stack**; `JAL`/`JALR` salvano l'indirizzo di ritorno in **`R31`**; **non c'è un registro dei flag**: le condizioni si scrivono in un registro con le `SET`.

2. Come si arriva a un indirizzo a 32 bit: `LHI` + offset

La decisione. Il DLX ha **un'unica modalità di indirizzamento: indiretto** [sl. 26]. L'indirizzo è sempre *registro (32 bit) + immediato (16 bit) esteso con segno*:

```
LW R7,Imm16(R8)     ; R7 ←32 M[R8 + (Imm16_15)^16 ## Imm16_15..0]      [sl. 26]
```

In parole `[aggiunto il 2026-11-05, su domanda di Lorenzo]`: il registro **tra parentesi** dà sempre l'**indirizzo**; quello **a sinistra** è il **dato** (nelle load ci arriva, nelle store ne parte). `L` = dalla memoria al registro, `S` = dal registro alla memoria; `W/H/B` = 4/2/1 byte.

Istruzione	Calcola l'indirizzo	Va in memoria e…	Il dato va…
`LW R7,4(R8)`	R8 + 4	**legge** una word	**in R7**
`LB R7,4(R8)`	R8 + 4	legge un byte, esteso **con segno**	in R7
`LBU R7,4(R8)`	R8 + 4	legge un byte, esteso **con zeri**	in R7
`SW R7,4(R8)`	R8 + 4	**scrive** una word	**da R7** alla memoria
`SB R7,4(R8)`	R8 + 4	scrive un byte (gli 8 bit bassi di R7)	da R7 alla memoria

Non è «copiare un registro in un altro»: R8 non viene copiato, dice solo *dove* andare.

Per raggiungere un indirizzo fisso come `F0000008h` si mettono i **16 bit alti** in un registro con `LHI`, e i **16 bit bassi** nell'offset:

```
LHI Rd,Imm16        ; Rd = Imm16 ## 0000h                              [sl. 33]
LHI R1,8420         ; R1 = 84200000h
```

Il perché. Un indirizzo è di 32 bit, ma nell'istruzione di formato I c'è posto solo per 16 (§1). `LHI` riempie la metà alta e azzera la bassa; l'offset della load o store aggiunge la metà bassa. ⚠️ **`LHI` NON accede alla memoria** [sl. 33]: costruisce solo un numero in un registro. Distinzione da tenere separata (errore trasversale n. 1): `LHI` *prepara* l'indirizzo, `LW`/`SW` *vanno* in memoria.

⚠️ **Conseguenza dell'estensione con segno** (sl. 26, ragionata): l'offset è un numero **con segno** a 16 bit, quindi da `0000h` a `7FFFh` somma, da `8000h` a `FFFFh` **sottrae**. Gli offset piccoli (`0x0008`, `0x0040`, `0x0800`) non creano problemi. Se servono i 16 bit bassi ≥ `8000h`, attenzione al verso (errore trasversale n. 5).

3. Quale load: dimensione e segno del dato

La decisione. Il registro è sempre di 32 bit, il dato in memoria no: byte, half-word o word [sl. 16]. Un dato più corto va **esteso a 32 bit** caricandolo, e si sceglie se tenerne il segno [sl. 16–17]:

Istruzione	Cosa carica	Come estende	Notazione del docente [sl. 30]
`LW`	word	—	`R1 ←32 M[40+R3]`

Istruzione	Cosa carica	Come estende	Notazione del docente [sl. 30]
LB / LH	byte / half-word	**con segno** (replica il bit 7 / 15)	`R1 ←32 (M[40+R3]_7)^24 ## M[40+R3]`
LBU / LHU	byte / half-word	**senza segno** (zeri)	`R1 ←32 (0)^24 ## M[40+R3]`
SW SH SB	scrivono 32 / 16 / 8 bit	—	`SB R1,0x0040(R3)`

Il perché. Esempio della dispensa [sl. 17]: `10110101` letto **unsigned** diventa `(0)^24##10110101`, letto **signed** `(1)^24##10110101`. Stesso byte, due numeri diversi (+181 e −75): la scelta dipende da **cosa dice il testo** del dato, non dal dato.

⚠️ **Distinzione da tenere separata** (trasversale n. 1): `LB` e `LBU` leggono lo **stesso byte** dal bus; cambia solo cosa finisce nei 24 bit alti del registro. Caso limite che le separa: un byte con il bit 7 a `1`.

4. Confrontare e saltare senza flag

La decisione. Non c'è un «salta se maggiore»: si fa in due istruzioni [sl. 32]. 1. `Sx Ra,Rb,Rc` (o `SxI`) mette `Ra` a 1 se la condizione è vera, altrimenti a 0 (x = `EQ, NE, LT, GT, LE, GE`) [sl. 29]; 2. `BEQZ/BNEZ` salta se quel registro è zero / non zero [sl. 32].

Il perché. Senza registro dei flag le condizioni vivono in un registro GP qualsiasi [sl. 14]. Il salto è relativo: `PC = PC + 4 + offset esteso con segno` [sl. 31–32].

La notazione — ciclo della dispensa, somma di 8 word da `00000800h` a `E0000200h` [sl. 49–50]:

```
        ADD    R1,R0,R0       ; azzera R1, accumulatore
        ADDI   R2,R0,20h      ; R2 = 32 (8 word × 4 byte)
Loop:   SUBI   R2,R2,4h       ; R2 = R2 - 4
        LW     R3,0800(R2)    ; legge word in memoria
        ADD    R1,R1,R3       ; aggiorna accumulatore R1
        BNEZ   R2,Loop        ; salta se R2 non è zero
        LHI    R7,0xE000      ; R7 = E0000000
        SW     R1,0200h(R7)   ; memorizza accumulatore in E0000200h
```

Due idee da portare via: `ADD R1,R0,R0` azzera usando `R0 = 0`; l'indice scende di **4** perché le word sono di 4 byte, e lo stesso registro fa da contatore e da offset.

5. Leggere e scrivere un dispositivo: è solo un indirizzo

La decisione. Una porta, un LED, un segnale come `STARTUP` sono **mappati** nello spazio di indirizzamento come le memorie [sl. 35]. Il software li legge con una load e li scrive con una store **al loro indirizzo**: non esistono istruzioni speciali di I/O.

Il perché, sul bus (collegamento con la 02). Una `LW` all'indirizzo di un dispositivo mette l'indirizzo su `BA[31..2]/BE`, alza `MEMRD`, e la decodifica accende **solo** il CS di quel dispositivo,

che risponde su BD [sl. 13, 40]. Per questo l'errore di stamattina (OUT dentro la RAM) era grave: una SW a quell'indirizzo avrebbe scritto **sia** nella porta **sia** nella RAM.

La notazione. Nell'esercizio della dispensa lettura e scrittura dello stesso segnale hanno **due chip-select e due indirizzi** [sl. 37]:

```
CS_READ_LED      =  (0x90000000)        CS_READ_STARTUP  =  (0xC0000000)
CS_SWITCH_LED    =  (0x90000004)        CS_WRITE_STARTUP =  (0xC0000004)
```

Le reti che realizzano LED e STARTUP (sl. 38–39) e il codice (sl. 41–42) sono lasciati vuoti nella dispensa: si fanno a lezione e tornano nei moduli 04–05.

Nella prova — 17/01/2023, consegna d)

Il testo: «Quanto letto dalla/e porta/e in input dovrà essere scritto, **come word**, all'indirizzo 0xF0000008»; «i registri da **R20 a R25** non debbano essere ripristinati».

La soluzione ufficiale [fonte: soluzione 2023-01-17, p. 9]:

```
00000000: LHI R20,0x6000      ; R20 = 60000000h
00000004: LW  R21,0x0000(R20) ; legge in R21 una word a 60000000h
00000008: LHI R22,0xF000      ; R22 = F0000000h
0000000C: SW  R21,0x0008(R22) ; scrive R21 a F0000008h
00000010: RFE
```

Il ragionamento, riga per riga. - 60000000h è CS_PORT, l'indirizzo che la soluzione dà alle quattro porte in input (consegna b, p. 2). Il codice usa **l'indirizzo deciso nella mappa**: mappa e codice sono la stessa decisione vista da due lati. - LHI + offset 0x0000: §2. I 16 bit alti 6000 vanno in LHI, i bassi (0000) nell'offset. - **LW, non LB**, anche quando si legge solo INPUT_PORT_0: la soluzione ha una rete che, quando 32_BIT = 0, mette sui fili BD[31..8] la copia del bit BD7, cioè (BD7)^8 su ogni byte alto (p. 7). È **l'estensione del segno di §3 fatta in hardware**, perché il byte è *signed*. Così l'handler fa sempre la stessa LW e non deve controllare 32_BIT con un branch. È la risposta alla consegna c) («l'handler il più veloce possibile»). - F0000008h sta nella RAM (80000000h–FFFFFFFFh): una SW lì accende CS_RAM_2GB_0…3, tutti e quattro perché è una word. - Si usano solo R20–R22, quindi niente da salvare (il testo lo concede). RFE chiude l'handler (modulo 04).

Criticità segnalata: leggere una word anche nel caso di un byte solo. Il costo è una rete in più, il guadagno sono istruzioni in meno nell'handler.

Passo di procedura

(Da riportare nel prontuario, §1, dopo il passo «Memorie».)

2. **Codice** *(lezione 03)*. (a) Per ogni indirizzo fisso che il codice usa: LHI Rx,<16 bit alti>, poi load/store con offset = 16 bit bassi (se ≥ 8000h sottrae: §2). (b) Scegli l'istruzione dal testo: word → LW/SW; byte o half-word *signed* → LB/LH; *unsigned* → LBU/LHU. (c) Usa solo i registri che il testo concede (es. R20–R25), R0 vale sempre 0.
    d. Confronti: Sx in un registro, poi BEQZ/BNEZ. (e) Gli indirizzi nel codice sono **gli stessi** della mappa della consegna b): ricontrollali lì.

Riepilogo

Perché per scrivere a `F0000008h` servono due istruzioni e non una? Nell'istruzione c'è spazio per un immediato di 16 bit. `LHI R22,0xF000` mette i 16 bit alti nel registro, `SW R21,0x0008(R22)` aggiunge i bassi come offset.

Il testo dice «byte di tipo *signed*»: cosa cambia nel codice? Si legge con `LB` (estende replicando il bit 7), non `LBU` (estende con zeri). Oppure, come nella 2023-01-17, una rete estende il segno sui fili e il codice legge sempre con `LW`.

Come fa il software a «parlare» con una porta? Con una load o store al suo indirizzo. La decodifica della consegna b) accende solo il suo CS, quindi se due dispositivi condividono un indirizzo, il codice li tocca tutti e due.